
Requirements, Design and Tasks for **rxnet**

Reactive execution networks

2026-03-11

Contents

Requirements Document	5
Introduction	5
Glossary	5
Requirements	6
Requirement 1: Shared Synchronous Phase Model	6
Requirement 2: Context and Input Snapshot Handling	6
Requirement 3: Deferred Action Execution	6
Requirement 4: Generic Node Runtime API	7
Requirement 5: FSM Semantics	7
Requirement 6: FSM Input Projection in C	8
Requirement 7: Petri Net Semantics	8
Requirement 8: PN Data Validation and Error Behavior	9
Requirement 9: C Frontend Lifecycle and Memory Management	9
Requirement 10: Python Frontend API Surface	9
Requirement 11: Integration Patterns and Examples	10
Requirement 12: Execution Model Constraints	10

List of Figures

Requirements Document

Introduction

This document specifies the requirements for `rxnet`, a reactive synchronous runtime library that supports two model families: Finite State Machines (FSM) and Petri Nets (PN). The system provides a shared phase-based execution core and language frontends in C and Python.

The goal of this document is to capture what the library provides today, so future changes can be validated and reflected here first.

Glossary

- **RXNET_System:** The complete `rxnet` library surface in C and Python
- **Core_Runtime:** Shared synchronous execution engine with phase-based ticking
- **Context:** Execution context containing live inputs, latched inputs, and deferred actions
- **Node:** Runtime unit with `evaluate` and `commit` phases
- **Deferred_Action:** Action scheduled during commit and executed after all commits
- **FSM_Runtime:** FSM frontend wrapper over the core runtime
- **FSM_Machine:** State machine node with transitions, guards, and optional actions
- **FSM_Transition:** Rule from `from_state` to `to_state`, with optional guard/action
- **Inputs_Projector:** C-only hook to map shared latched inputs into machine-local user data before FSM guard evaluation
- **PN_Runtime:** Petri Net frontend wrapper over the core runtime
- **PN_Net:** Petri net node with places, transitions, and firing flags
- **PN_Transition:** Petri transition with consume/produce arcs, optional guard/action
- **Arc:** Petri arc with `place_id` and `weight`
- **Tick:** One complete synchronous cycle: latch -> evaluate -> commit -> deferred actions
- **Latched_Inputs:** Immutable snapshot of current inputs used during one tick

Requirements

Requirement 1: Shared Synchronous Phase Model

User Story: As a model developer, I want deterministic phase-based execution, so that all nodes observe a consistent input snapshot and state updates happen predictably.

Acceptance Criteria

1. WHEN a tick starts, THE Core_Runtime SHALL latch input data before node evaluation
2. WHEN latching completes, THE Core_Runtime SHALL evaluate all registered nodes
3. WHEN evaluation completes, THE Core_Runtime SHALL commit all registered nodes
4. WHEN commit completes, THE Core_Runtime SHALL execute deferred actions and clear the queue
5. THE Tick order SHALL remain `latch -> evaluate -> commit -> deferred actions` in both C and Python implementations

Requirement 2: Context and Input Snapshot Handling

User Story: As an integrator, I want separate live and latched inputs, so that guards read a stable snapshot during each tick.

Acceptance Criteria

1. THE Context SHALL expose mutable live inputs for application/driver updates
2. THE Context SHALL expose latched inputs used during evaluation
3. WHEN ticking in C, THE runtime SHALL copy `inputs_size` bytes from `inputs` to `latched_inputs`
4. WHEN ticking in Python, THE runtime SHALL update `latched_inputs` using `copy.copy(inputs)`
5. THE C implementation SHALL keep ownership of the live inputs buffer in the application (runtime does not free it)

Requirement 3: Deferred Action Execution

User Story: As a runtime user, I want actions to run after all commits, so that side effects are deferred and model state commits are consistent.

Acceptance Criteria

1. WHEN a node commits with a proposed action, THE Context SHALL enqueue the action for deferred execution
2. THE Core_Runtime SHALL run deferred actions only after all node commits in the tick
3. THE deferred action queue SHALL be cleared after execution
4. THE C implementation SHALL grow deferred action storage dynamically when capacity is exceeded
5. WHEN enqueue parameters are invalid in C (`ctx` or function is null), THE API SHALL return an error code (-1)

Requirement 4: Generic Node Runtime API

User Story: As a library user, I want a generic runtime abstraction, so that different model families can share one execution engine.

Acceptance Criteria

1. THE Node contract SHALL include `evaluate` and `commit` callbacks
2. THE C runtime SHALL support runtime initialization with explicit node capacity
3. WHEN adding nodes in C beyond configured capacity, THE runtime SHALL reject the operation with -1
4. THE Python runtime SHALL support dynamic node list growth without explicit capacity configuration
5. WHEN `tick` is called with invalid runtime/context in C, THE API SHALL return -1

Requirement 5: FSM Semantics

User Story: As an FSM author, I want first-match transition evaluation with deferred actions, so that machine behavior is deterministic and clear.

Acceptance Criteria

1. WHEN an FSM machine evaluates, THE machine SHALL reset `next_state` to current state and clear proposed action
2. THE FSM evaluator SHALL scan transitions in declaration order and select the first transition whose `from_state` matches and whose guard is true (or absent)

3. WHEN no transition matches, THE machine SHALL remain in the current state
4. WHEN committing, THE machine SHALL update current state to [next_state](#)
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action (not executed inline during evaluate/commit)

Requirement 6: FSM Input Projection in C

User Story: As an embedded integrator, I want optional per-machine input projection, so that multiple machines can share one global input snapshot while evaluating machine-local signals.

Acceptance Criteria

1. THE C FSM API SHALL support configuring an optional [inputs_projector](#) callback per machine
2. WHEN [inputs_projector](#) is configured, THE callback SHALL execute at the start of machine evaluation
3. THE projector SHALL receive the shared latched context and machine user payload
4. THE runtime SHALL allow multiple FSM machines to share one context input struct
5. IF no projector is configured, THEN FSM evaluation SHALL proceed directly with transition guard checks

Requirement 7: Petri Net Semantics

User Story: As a Petri Net author, I want place/transition execution with guards and arc validation, so that token flow is modeled consistently.

Acceptance Criteria

1. WHEN a PN net evaluates, THE net SHALL copy [places](#) into [next_places](#) and reset transition fire flags
2. THE PN evaluator SHALL mark a transition as fireable only if consume/produce arcs are valid and consume tokens are available
3. WHEN a transition guard exists, THE transition SHALL fire only if the guard returns true
4. WHEN committing, THE net SHALL apply consume/produce deltas for each fire-flagged transition
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action

Requirement 8: PN Data Validation and Error Behavior

User Story: As a developer, I want invalid Petri model data to be detected, so that configuration errors fail early.

Acceptance Criteria

1. WHEN initializing a PN net in C, THE API SHALL validate all consume/produce arc lists and reject invalid models with `-1`
2. THE C PN arc validator SHALL reject arcs with out-of-range `place_id` or negative `weight`
3. WHEN PN net allocation fails in C, THE API SHALL free partial allocations and return `-1`
4. IN Python PN evaluation, invalid arc `place_id` SHALL raise `IndexError`
5. IN Python PN evaluation, negative arc weights SHALL raise `ValueError`

Requirement 9: C Frontend Lifecycle and Memory Management

User Story: As a C integrator, I want explicit initialization and cleanup APIs, so that runtime memory is managed safely.

Acceptance Criteria

1. THE C API SHALL provide runtime init/free functions for both FSM and PN frontends
2. THE C API SHALL provide model registration functions (`add_machine`, `add_net`) before ticking
3. THE C API SHALL provide model-specific cleanup for PN net internal buffers (`rx_pn_net_free`)
4. WHEN freeing runtime/context in C, THE API SHALL reset internal pointers and counters to neutral values
5. WHEN null pointers are passed to C free functions, THE functions SHALL return safely without crashing

Requirement 10: Python Frontend API Surface

User Story: As a Python developer, I want ergonomic dataclass-based APIs, so that models are easy to define and run.

Acceptance Criteria

1. THE Python FSM frontend SHALL expose `Machine` and `Transition` dataclasses with optional guard/action callbacks
2. THE Python PN frontend SHALL expose `Net`, `Transition`, and `Arc` dataclasses
3. THE Python runtime wrappers (`rxnet.fsm.Runtime`, `rxnet.pn.Runtime`) SHALL expose `context`, model registration, and `tick()`
4. THE root package SHALL expose `fsm`, `pn`, `runtime`, and FSM-oriented aliases currently defined in `rxnet.__init__`
5. THE Python implementation SHALL rely only on standard-library modules present in the code-base (`dataclasses`, `typing`, `copy`)

Requirement 11: Integration Patterns and Examples

User Story: As a user evaluating the library, I want runnable examples in C and Python, so that I can understand expected integration patterns.

Acceptance Criteria

1. THE repository SHALL include runnable C examples for FSM and PN under `c/examples`
2. THE repository SHALL include runnable Python examples for FSM and PN under `python/examples`
3. THE example structure in Python SHALL separate model definition, system input writer, and runtime wiring (`model.py`, `system.py`, `main.py`)
4. THE repository SHALL include an ESP-IDF FSM integration example with periodic ticking and ISR-driven input updates
5. THE host CLI FSM example (`c/examples/fsm/00-light/main_cli.c`) SHALL support interactive commands to trigger input events and ticks

Requirement 12: Execution Model Constraints

User Story: As a platform engineer, I want clear operational constraints, so that I can integrate `rxnet` correctly in larger systems.

Acceptance Criteria

1. THE runtime SHALL be single-threaded per runtime instance unless external synchronization is provided by the integrator
2. THE library SHALL not include built-in networking, persistence, or REST APIs
3. THE library SHALL leave scheduling policy (tick frequency/loop ownership) to the host application
4. THE library SHALL leave I/O side effects to user-defined action callbacks
5. THE library SHALL support using one shared typed input structure across multiple model nodes in the same runtime context